

MALICIOUS URL DETECTION USING MACHINE LEARNING

Group Number: 19

Group Members

Aarya Balwadkar

Afifa Bintul Hasan

Aparna Nair

Eccha Bansal

*Department of Computer Science / Computer Engineering
Symbiosis Institute of Technology, Pune | Academic Year 2026–27*

Abstract

The widespread use of web-based services has established the Uniform Resource Locator (URL) as a primary vector for phishing, malware delivery, spam and related forms of online fraud. Blacklist-based defences remain effective against previously identified threats, yet they rely on prior observation of a URL or domain and therefore offer limited protection against newly created or slightly altered malicious links. This case study investigates the use of machine learning for malicious URL detection, concentrating on lexical and structural attributes that can be derived directly from the URL string without retrieving the target page. Relevant research on lexical, host-based and hybrid representations is reviewed, together with the characteristics of established

classical algorithms, namely Logistic Regression, Decision Tree, Random Forest, Support Vector Machine, K-Nearest Neighbours and gradient-boosted ensembles. The ISCX-URL2016 collection, a publicly documented multi-class dataset covering benign, spam, phishing, malware and defacement URLs [4], has been selected for the proposed work. The methodology presented here addresses dataset acquisition, cleaning, exploratory analysis, URL normalisation, feature extraction, feature selection, stratified partitioning and evaluation with Accuracy, Precision, Recall, F1-score and ROC-AUC. No practical implementation was undertaken and no original performance figures are reported; the study instead provides a justified methodological design and a literature-based assessment of how machine-learning models may supplement existing URL-reputation mechanisms in browser, email and enterprise settings.

Keywords

Malicious URL Detection; Phishing; Machine Learning; Lexical Features; Feature Engineering; Cybersecurity; Random Forest; XGBoost; URL Classification; Web Security.

1. Introduction

1.1 Background

A Uniform Resource Locator (URL) provides the standard means of addressing resources on the World Wide Web. Its structure normally comprises a scheme such as http or https, a hostname or domain, an optional subdomain, a path, a query component and, in some cases, a port number or fragment identifier. Since users depend on URLs to reach web content, the same mechanism provides attackers with a direct means of directing victims to hostile infrastructure.

A URL is regarded as malicious where it guides a user, whether directly or through redirection, to content intended to cause harm. The literature generally distinguishes the following categories according to the form of harm involved:

- Phishing URLs — pages that imitate a trusted organisation in order to obtain login credentials, payment details or other confidential information.
- Malware-hosting URLs — links that cause the visitor device to receive malicious software, including viruses, worms, trojans, ransomware or spyware, frequently through concealed drive-by downloads.

- Spam URLs — links distributed through unsolicited bulk messages and generally promoting fraudulent, low-value or unwanted goods and services.
- Defacement URLs — links to sites whose content has been altered without authorisation, commonly to display an ideological message or to demonstrate a successful compromise.
- Scam URLs — pages constructed around social-engineering narratives, such as fictitious lotteries, false technical-support alerts or romance fraud, intended to secure financial gain.

1.2 URL Obfuscation and the Difficulty of Detection

Malicious links are seldom presented without concealment. Common obfuscation practices include substituting an IP address for a domain name, inserting an '@' character so that the browser disregards the preceding string during host resolution, adding multiple subdomains to suggest affiliation with a known brand (for example, secure-login.paypal.example.com), employing hexadecimal or percent-encoding to mask characters, or routing the link through a shortening service. Such measures allow a harmful URL to resemble a legitimate address and to bypass elementary pattern checks, which explains why detection continues to present a substantive classification challenge.

1.3 Limitations of Traditional Blacklist and Whitelist Approaches

The most widely deployed defence remains the blacklist, namely a maintained list of URLs or domains previously confirmed as malicious and compiled from user reports, honeypots and vendor telemetry; services such as Google Safe Browsing follow this principle. A whitelist adopts the opposite approach by permitting only approved domains. Although blacklists are straightforward to implement, inexpensive to query and reliable for known entries, the literature identifies several inherent weaknesses:

- Blacklists operate retrospectively. A URL must have been observed, reported and verified before inclusion, leaving a period during which it can be used against victims without restriction.
- Attackers can register large numbers of temporary domains or introduce minor variations to listed URLs, thereby avoiding static entries at limited cost.
- Blacklists do not generalise. A previously unseen URL receives no assessment, even where its structure closely resembles known malicious examples.
- Maintenance and distribution of such lists at Internet scale entail storage, latency and update-propagation costs.

1.4 The Role of Machine Learning in URL Classification

Machine learning approaches the task as a general classification problem rather than as a lookup operation. Instead of determining whether a particular URL has been encountered before, a trained model identifies statistical and structural regularities that separate malicious from legitimate addresses, for instance atypical length, uncommon character distributions, unusual subdomain depth or the occurrence of suspect terms. A model of this kind can therefore assign a risk score to a previously unseen address, which permits real-time or near-real-time assessment of zero-day URLs. For this reason, machine learning is best viewed as supplementary to blacklist filtering. Contemporary security products commonly combine the two methods, using the blacklist as a rapid initial check and the model to address novel cases. Where classification relies solely on lexical properties of the URL string, no retrieval, rendering or execution of the destination is required, which preserves both speed and safety, while reducing reliance on manually maintained signatures.

2. Relevance of the Topic in Today's World

Detection of malicious URLs lies at the convergence of two sustained developments: the extensive reliance of daily activities on web applications, and the wider adoption of machine learning within cybersecurity practice. To maintain a clear distinction between evidence and analysis, this section presents verified industry statistics separately from published research findings, followed by the interpretation of the project group.

2.1 Established Facts — Scale of the Threat

The Anti-Phishing Working Group (APWG), an industry body monitoring phishing activity since 2003, recorded 3.8 million distinct phishing sites during 2025, compared with 3.76 million in 2024, including 853,244 distinct sites in the fourth quarter of 2025 alone. Its quarterly reports further indicate that SaaS and webmail services together with social media remain the most frequently impersonated sectors, and that business email compromise involving wire transfers increased by 57% between quarters in the third quarter of 2025, with a mean requested amount of USD 48,115. Separately, the Internet Crime Complaint Center (IC3) of the United States Federal Bureau of Investigation reported 193,407 complaints relating to phishing and spoofing in 2024, alongside confirmed business email compromise losses of USD 2.77 billion. Taken from the organisations'

own published reports, these figures confirm that URL-based offending remains a large-scale and, in several respects, expanding problem.

2.2 Findings Reported by Researchers

A considerable body of published research, examined in Section 5 of this report, has shown that lexical and structural properties of the URL alone can support accurate classification without accessing the destination page. The survey by Sahoo, Liu and Hoi formalises the task as a learning problem and defines the lexical, host-based and content-based feature groups adopted by later studies. Work published between 2022 and 2025 further reports that ensemble tree methods, particularly Random Forest and XGBoost, frequently achieve results comparable to deep-learning alternatives on tabular URL features while requiring markedly less computation. This explains why classical methods continue to receive attention. The same literature also records increasing concern with concept drift and adversarial construction of URLs intended to evade trained classifiers, which underlines the need for robust and regularly updated models.

2.3 Our Interpretation

The industry scale data together with the research findings indicate that malicious URL detection constitutes a practical cybersecurity concern rather than a theoretical exercise. Wider use of online banking, electronic commerce, remote working and cloud-based collaboration has increased the number of unfamiliar links encountered each day through email, messaging, QR codes and social platforms, all of which feature in APWG reporting as delivery channels. A method that can assess a previously unseen URL promptly, operate within browser or mail-gateway constraints, and be retrained economically as attack patterns change therefore responds to an existing operational requirement. The group regards the topic as relevant in application, since it applies an efficient learning pipeline to a rapidly evolving threat, while remaining firmly situated within established research in machine learning and web security.

3. Problem Statement

In view of the constraints of static blacklist and whitelist filtering outlined in Section 1, a classification method is required that can separate malicious from legitimate URLs on the basis of properties contained within the URL itself, so that unfamiliar addresses can still be evaluated with

acceptable confidence and speed. The central question for this case study is therefore how machine-learning models trained on measurable lexical and structural URL attributes can distinguish malicious URLs from legitimate ones, while maintaining false-negative and false-positive rates at levels acceptable for practical use.

The scope of the claim should be stated explicitly. The system described here is not advanced as a complete response to cyber threats, nor as a substitute for layered controls such as blacklists, browser isolation, email authentication standards (SPF, DKIM and DMARC), user-awareness programmes or endpoint protection. It is proposed as an additional layer directed at the specific weakness of list-based methods, namely the handling of newly created or previously unobserved malicious URLs through generalisation from learned patterns.

4. Objectives

The study pursues the following specific objectives:

- To examine and consolidate published research on lexical, host-based and hybrid representations for malicious URL detection, and to identify the feature groups and algorithms reported to perform effectively.
- To define a set of lexical and structural URL attributes capable of separating malicious from legitimate addresses without retrieving or executing the destination page.
- To specify a reproducible preprocessing workflow for a malicious-URL dataset, addressing duplicate removal, missing-value treatment, URL normalisation and label encoding.
- To derive a concise and non-redundant collection of lexical and suspicious-indicator features from raw URL strings for model input.
- To train and compare several classical classifiers — Logistic Regression, Decision Tree, Random Forest, Support Vector Machine, K-Nearest Neighbours and a gradient-boosted ensemble such as XGBoost — using a common feature set.
- To evaluate the resulting models with Accuracy, Precision, Recall, F1-score and ROC-AUC, with particular consideration of the false-negative rate in view of its security implications.

- To assess the practical value, limitations and deployment factors associated with a lexical-feature approach to malicious URL detection, and to outline reasonable directions for further work.

5. Literature Review

This section reviews fifteen studies from the peer-reviewed and preprint literature on malicious URL and phishing detection, ranging from early work published in 2009 to survey articles appearing in 2025. The discussion is organised by feature strategy — lexical-only, hybrid, host-oriented and deep-representation methods — and Table 1 provides a comparative summary. Complete bibliographic information for each cited work is given in the References (Section 21) under IEEE numbering.

5.1 Foundational and Survey Work

Ma, Saul, Savage and Voelker [1] were among the earliest to treat malicious website detection as a statistical learning task. They showed that lexical and host-based URL attributes, available without retrieving the target page, could be combined with online learning methods to identify suspect addresses, a position described as moving 'beyond blacklists'. Their formulation continues to underpin much subsequent work on lexical features.

Sahoo, Liu and Hoi [2] subsequently provided a broad survey that organises the field according to four feature groups — lexical, host-based, content-based and other — and according to the use of classical learning as against representation learning. The authors observe that lexical attributes are economical to compute and avoid contact with the destination server, although models relying on them alone may be more susceptible to deliberately crafted evasive URLs than those incorporating host or content evidence.

A survey published in 2025 [3] extends this classification to contemporary representation methods, including Transformer architectures and graph neural networks, and structures the field by data modality, covering lexical strings, HTML, JavaScript behaviour and visual information. The authors note that classical learning on lexical features remains a competitive low-latency reference point in production-oriented studies despite progress in deep methods.

5.2 Lexical-Feature-Based Classical Machine Learning Studies

Mamun, Rathore, Lashkari, Stakhanova and Ghorbani [4] introduced the ISCX-URL2016 collection and showed that lexical attributes combined with classifiers including Random Forest could separate benign addresses from phishing, malware, spam and defacement categories. The dataset has since been widely adopted as a public benchmark.

Joshi, Lloyd, Westin and Seethapathy [5] constructed a 1,023-dimensional URL representation from 23 manually engineered lexical attributes together with 1,000 trigram-based Word2Vec-type features. After removing one member of each feature pair correlated above 0.75, they trained a Random Forest ensemble and reported strong discriminative results using only URL-derived information, without host or content signals.

Vanhoenshoven, Napoles, Falcon, Vanhoof and Koppen [6] compared several classical classifiers, including Naive Bayes, k-Nearest Neighbours and tree-based ensembles, using purely lexical URL properties. Ensemble tree methods obtained superior results to the simpler probabilistic alternatives on their data, a pattern repeated in most later lexical studies considered here.

Sahingoz, Buber, Demir and Diri [7] combined word-level natural-language features with seven classification algorithms and released a substantial collection termed the 'Big URL Data Set', comprising 36,400 legitimate and 37,175 phishing URLs. Their Random Forest configuration based on natural-language and word-vector features achieved a reported accuracy of 97.98%, and the study remains a frequently cited reference for phishing classification from URL text.

Aljabri, Altamimi and co-authors [8] compared lexical, network-based and content-based feature groups across classical and deep models. Their results indicated that combination of groups generally improved on any single group, while lexical attributes supplied the greatest discriminative contribution relative to computational cost.

5.3 Feature Engineering and Domain/Host-Level Studies

Kara, Ok and Ozaday [9] concentrated on URL and domain-name properties, such as domain length, character composition and the use of uncommon top-level domains. Evaluated with standard classifiers, these attributes provided dependable separation of phishing from legitimate sites, which supports the value of careful domain-level engineering.

Gualberto, De Sousa, Thiago, Da Costa and Duque [10] proposed supplementing conventional lexical attributes with features derived from topic modelling. Their argument is that unsupervised topic extraction from URL tokens, combined with established engineering practice, can improve detection beyond either source considered separately.

Innab, Osman, Ataelfadiel, Abu-Zanona, Elzaghmouri, Zawaideh and Alawneh [11] assessed an ensemble of learning algorithms for phishing detection. Their findings align with the wider pattern observed in this review, namely that ensemble and boosted-tree approaches generally surpass single-classifier baselines on tabular URL features.

5.4 Hybrid and Deep-Representation Studies

Li, Yang, Chen, Yuan and Liu [12] presented a stacking ensemble that unites URL-derived and HTML-derived attributes. The addition of limited page-content information improved on lexical-only accuracy, although fetching the page necessarily increases latency and requires retrieval of potentially unsafe material.

Somesha, Pais, Rao and Rathour [13] compared several deep architectures for phishing-site detection with classical baselines. Deep models matched or exceeded classical accuracy where sufficient data were available, yet classical ensembles remained strongly competitive and substantially less demanding in training and deployment.

Le, Pham, Sahoo and Hoi [14] introduced URLNet, an end-to-end architecture that derives a URL representation directly from character and word sequences with convolutional networks, thereby removing manual feature construction. The learned representation is reported to capture sequential and semantic regularities that hand-crafted attributes may overlook, at the expense of greater data and computational requirements. Related work by Saxe and Berlin [15] on the eXpose character-level convolutional network reports comparable conclusions for malicious URLs, file paths and registry keys from raw character input.

Opara, Chen and Wei [16] integrated raw URL character sequences with HTML-derived attributes within a unified deep pipeline and reported gains over URL-only baselines for phishing-page detection. Their results again illustrate the general trade-off between the additional accuracy of hybrid content-aware methods and the latency and safety costs of content retrieval.

5.5 Synthesis and Relevance to This Case Study

Three recurrent observations from the fifteen studies directly shape the methodology proposed here. First, lexical attributes derived solely from the URL offer considerable discriminative strength at minimal computational expense, which makes them appropriate as the foundation for a real-time system; studies incorporating host or content signals generally report further but comparatively modest improvements. Second, ensemble tree classifiers, notably Random Forest and gradient-boosted variants such as XGBoost, are repeatedly found to equal or surpass simpler classical methods and, in several comparisons, to remain competitive with far more demanding deep models on tabular URL data. Third, limited generalisation across datasets, together with susceptibility to concept drift and adversarial URL design, recurs as a limitation, so that accuracy measured on one collection cannot be assumed to transfer directly to other periods or sources. The proposed approach — broad lexical extraction, comparison of several classical and ensemble classifiers, and explicit treatment of generalisation and robustness as limitations — follows directly from these observations.

Table 1: Comparative summary of the literature reviewed on malicious URL detection

Ref.	Year	Dataset	Feature Type(s)	Model(s)	Key Finding	Limitation Noted
[1]	2009	Proprietary crawled URLs	Lexical + host-based	Online learning (classical)	Established feasibility of statistical URL classification without crawling destination content	Proprietary, non-public dataset limits reproducibility
[2]	2017/19	Multiple (survey)	Lexical, host-based, content-based	Survey of classical + deep methods	Formalised feature taxonomy still used by later work	Survey — no new experimental results
[3]	2025	Multiple (survey)	Lexical, HTML, JS, visual, graph	Survey incl. Transformer/GNN	Classical lexical-ML remains a strong, low-latency baseline vs. deep methods	Newer modalities (visual/GNN) add cost and complexity
[4]	2016	ISCX-URL2016	Lexical	Random Forest and others	Established a widely reused public multi-class URL benchmark	Dataset reflects only 2016-era URL patterns (potential drift)
[5]	2019	Custom crawled corpus	Lexical + trigram/Word2Vec	Random Forest	Purely lexical + n-gram features gave strong discrimination	High-dimensional trigram features increase training cost
[6]	2016	Custom dataset	Lexical	Naive Bayes, k-NN, ensembles	Ensemble tree methods outperformed simpler classifiers	Small-scale evaluation; limited generalisability discussed
[7]	2019	Big URL Data Set (73,575 URLs, self-released)	NLP word-vector + lexical	7 classifiers incl. Random Forest	97.98% accuracy reported with NLP + Random Forest	Single-source dataset; possible sampling bias
[8]	2022	Multiple public datasets	Lexical, network, content	Classical ML + deep learning	Combining feature categories improved accuracy over single category	Content/network features require fetching destination site

Ref.	Year	Dataset	Feature Type(s)	Model(s)	Key Finding	Limitation Noted
[9]	2022	URL/domain feature dataset	Domain & lexical features	Classical ML classifiers	Domain-name-level lexical features reliably separated phishing from legitimate sites	Focuses narrowly on domain features; excludes path/query signals
[10]	2020	Custom phishing corpus	Lexical + topic-model features	Classical ML	Topic-modelling features added incremental accuracy gains	Added preprocessing complexity and interpretability cost
[11]	2024	Public phishing dataset	Lexical/derived features	Ensemble ML algorithms	Ensemble methods outperformed single-classifier baselines	Evaluated on a single dataset; generalisation untested
[12]	2019	Custom URL+HTML corpus	Lexical + HTML	Stacking ensemble	Adding HTML features improved on URL-only accuracy	Requires fetching page content, adding latency and risk
[13]	2020	Public phishing datasets	Lexical/derived	Deep learning vs. classical ML	Classical ensembles remained competitive with deep models	Deep models needed larger data volumes to match ensembles
[14]	2018	Public + proprietary URL corpora	Raw character/word sequence (learned)	CNN (URLNet)	Learned representations captured patterns manual features missed	Higher computational cost; less interpretable than lexical features
[16]	2024	Public phishing corpus	Raw URL + HTML (learned)	Deep learning	Hybrid raw-URL + HTML model improved over URL-only baseline	Content fetch dependency; higher pipeline complexity

6. Methodology

The methodology follows a staged pipeline consistent with standard practice identified in Section 5, adapted to the constraints of an undergraduate case study and to the requirement of reproducibility. Each stage is described together with its purpose and its position in the sequence, since ordering bears directly on validity; scaling, selection and resampling parameters, for example, must be estimated from training data only to prevent leakage, as addressed in Section 8.

6.1 Overview of the Pipeline

The complete workflow comprises twelve stages: (1) acquisition of the dataset, (2) data cleaning, (3) exploratory analysis, (4) URL preprocessing and normalisation, (5) feature extraction, (6) feature selection, (7) partitioning into training, validation and test sets, (8) model selection, (9) model training, (10) prediction on held-out data, (11) evaluation with several metrics, and (12) comparison and interpretation across models.

6.1.1 Dataset Acquisition

The workflow starts with collection of a labelled URL set in which each entry is marked as benign or malicious, or, in the multi-class formulation, assigned to a malicious sub-category such as

phishing, malware, spam or defacement. Candidate sources and the dataset adopted for this study are detailed in Section 7.

6.1.2 Data Cleaning

URL collections assembled from blacklist feeds, crawl logs and community phishing reports commonly contain duplicates, truncated or malformed entries, absent labels and inconsistent encodings. This stage removes exact and near-duplicate records, discards or imputes entries with missing critical fields, and standardises encoding so that later feature computation operates on uniform input. Cleaning precedes exploratory analysis to ensure that summary statistics are not distorted by corrupted records.

6.1.3 Exploratory Data Analysis (EDA)

Exploratory analysis characterises class proportions, URL-length distributions, top-level-domain frequencies and possible artefacts, for instance concentration of malicious entries within a narrow time interval or a single reporting source that might lead the model to learn source-specific traits rather than genuinely malicious structure. Findings at this point guide feature choices (Section 9) and indicate whether imbalance treatment will be needed (Section 8).

6.1.4 URL Preprocessing and Normalisation

Prior to numerical encoding, each URL string is divided into its structural parts — scheme, host, subdomain, path, query and fragment — using a standard parsing library. Normalisation is applied uniformly, including lower-casing of the scheme and host while retaining the original case of path and query strings, since case variation in those components can itself carry signal, alongside decoding of percent-encoded sequences for analytical purposes.

6.1.5 Feature Extraction

Each normalised URL is then mapped to a fixed-length numeric vector using the lexical and suspicious-indicator attributes specified in Section 9. This conversion restates the task from classification of variable-length strings to supervised learning over a fixed-dimensional table, to which classical classifiers can be applied directly.

6.1.6 Feature Selection

Extracted attributes differ in relevance and several length-based measures are strongly intercorrelated, as noted in the studies discussed in Section 5.2. Selection through correlation-

based pruning, importance ranking from a preliminary tree model, or variance thresholding is therefore used to limit redundancy, shorten training and reduce overfitting to correlated groups.

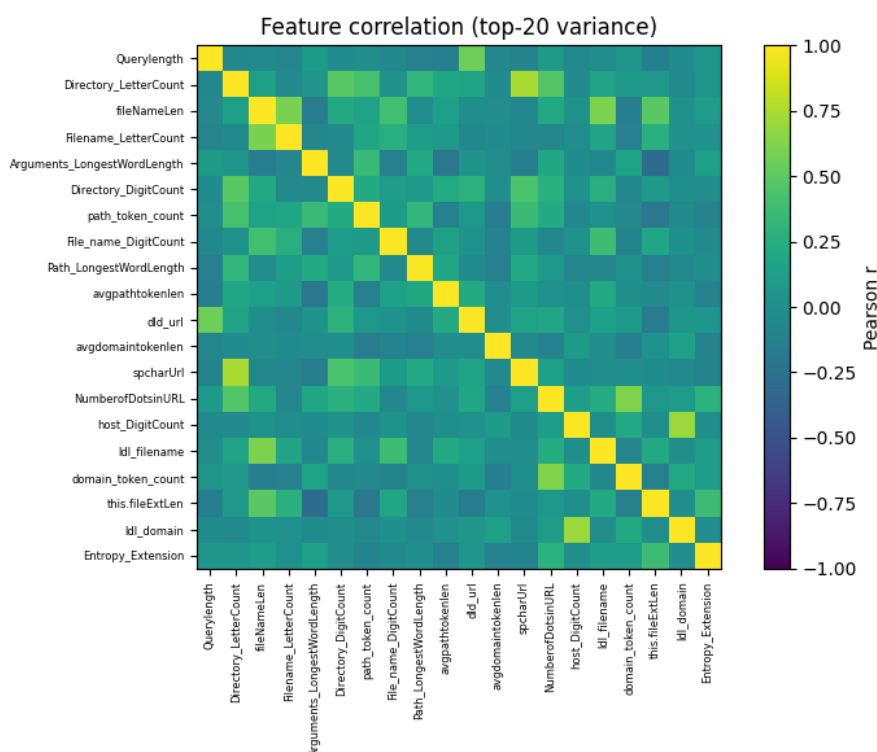


Figure 1: Pearson correlation among the top-20 highest-variance lexical features on the training set, motivating correlation pruning at 0.75 (original figure).

6.1.7 Train/Validation/Test Splitting

The cleaned and encoded collection is divided into training, validation and test portions; a 70:15:15 division is adopted here, which falls within the 70:15:15 to 80:10:10 range commonly reported. Stratified sampling by class label preserves the benign-to-malicious proportion in each portion. All scaling parameters, selection decisions and resampling operations are estimated solely from the training portion and subsequently applied without modification to the validation and test portions in order to prevent leakage (see Section 8.7).

6.1.8–6.1.12 Model Training, Prediction, Evaluation and Comparison

All classifiers described in Section 10 are trained separately on the same training portion and feature set, so that observed differences can reasonably be attributed to algorithm choice rather than data variation. Hyperparameters are adjusted using the validation portion through grid or randomised search with cross-validation, while final unbiased figures are taken exclusively from

the held-out test portion, which remains unseen during training and tuning. Model comparison then uses the metrics defined in Section 12, with discussion provided in the results and comparative sections.

7. Dataset Description

Dataset adopted: ISCX-URL2016, developed by the Canadian Institute for Cybersecurity, University of New Brunswick. The collection contains approximately 114,400 URLs across benign, spam, phishing, malware and defacement classes and was introduced by Mamun et al. [4].

7.1 Recommended Candidate Datasets

- ISCX-URL2016 (Canadian Institute for Cybersecurity, University of New Brunswick) — a multi-class labelled collection of benign, phishing, malware, spam and defacement URLs with precomputed lexical attributes, introduced by Mamun et al. [4] and widely used as a benchmark in the studies reviewed in Section 5.
- The 'Big URL Data Set' published by Sahingoz et al. [7], containing 36,400 legitimate and 37,175 phishing URLs assembled specifically for learning-based phishing research.
- PhiUSIIL and GramBeddings — two recent large-scale public phishing benchmarks cited in 2025 comparative studies, suitable where a larger and more current sample is preferred to lessen the effects of concept drift.
- Community-maintained Kaggle malicious-URL collections whose provenance can be confirmed, for instance sets citing PhishTank, URLHaus or OpenPhish for malicious entries combined with legitimate addresses from a ranked-domain list.

Whichever source is ultimately used, the following must be documented: total sample count, class definitions and counts, the precise benign-to-malicious ratio, original sources and collection period, which lexical or host-based attributes are supplied as against those computed from the raw string, and any quality concerns — mislabelled or duplicated entries, or class imbalance — identified by the original authors or through the group's own exploratory analysis (Section 6.1.3).

7.2 Selected Dataset

ISCX-URL2016, published by the Canadian Institute for Cybersecurity, University of New Brunswick, has been selected for this study. Its documentation defines five categories: benign, spam, phishing, malware and defacement. Benign entries were drawn from highly ranked Alexa sites and screened with VirusTotal, while malicious groups were assembled from sources including

WEBSHAM-UK2007, OpenPhish and DNS-BH [4]. The collection is appropriate here because the associated research directly examines lightweight lexical analysis for proactive detection [4].

7.3 Class Distribution and Imbalance

ISCX-URL2016 comprises approximately 114,400 URLs. The dataset description reports more than 35,300 benign addresses, about 12,000 spam, about 10,000 phishing, over 11,500 malware and over 45,450 defacement entries [4]. These values are stated as approximate or lower-bound counts rather than an exact partition. The presence of several malicious categories permits discussion of both binary malicious-versus-benign classification and multi-class categorisation.

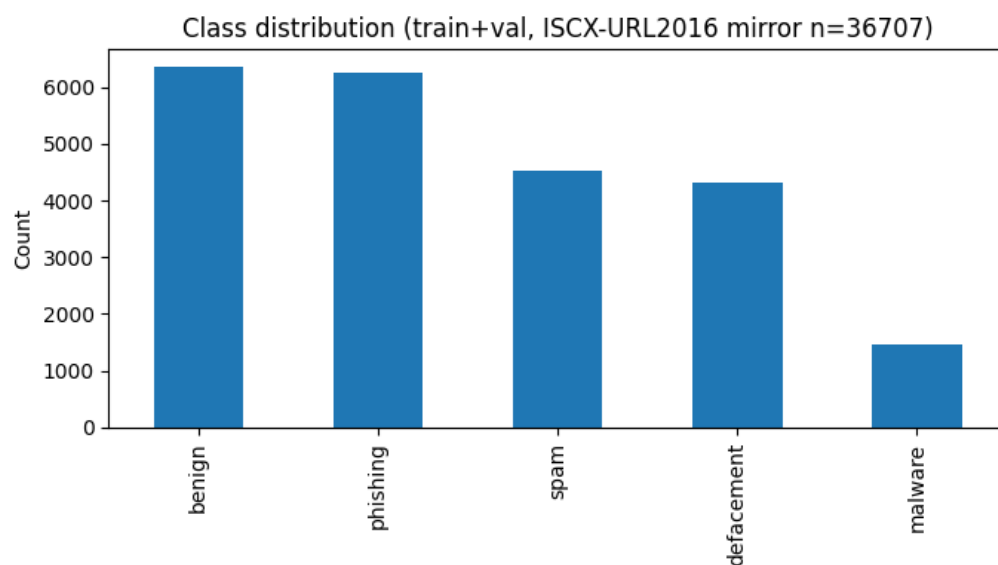


Figure 2: Class distribution after duplicate removal (n=26,953; defacement majority retained; original figure from project data).

8. Data Preprocessing

Preprocessing renders the raw labelled collection suitable for dependable feature computation and training. Each operation below is included for a stated reason rather than by convention.

8.1 Duplicate Removal

Exact and near-duplicate addresses, differing only in trailing slashes, tracking parameters or letter case of the scheme, are identified and removed. Retention of such duplicates would be problematic where the same address appears in both training and test portions, since measured accuracy would then reflect memorisation of seen entries rather than genuine generalisation to new URLs.

8.2 Missing-Value Handling

Entries lacking a valid label are removed entirely, as supervised training and evaluation cannot proceed without ground truth. Where auxiliary fields are absent, for example a missing WHOIS result for a host-based attribute, a documented rule is applied — either a placeholder 'unknown' category or exclusion of the dependent feature — and the choice is reported to preserve reproducibility.

8.3 URL Normalisation

Normalisation is applied so that inconsequential formatting differences do not yield distinct feature vectors for otherwise identical addresses. In particular, the scheme and hostname are lower-cased while path and query case is retained, and percent-encoded characters are decoded for the purpose of feature calculation.

8.4 Handling Malformed URLs

Addresses that the parsing library cannot interpret because of absent schemes, invalid characters or truncation are flagged and either corrected with a limited heuristic, such as supplying a default 'http://' scheme, or omitted from the collection. The number of omissions is recorded so that the effective sample size remains transparent.

8.5 Class Imbalance Handling

Where exploratory analysis (Section 6.1.3) shows marked imbalance between benign and malicious classes, resampling may be applied, including random undersampling of the majority class, oversampling of the minority class, or synthetic generation such as SMOTE. Without such treatment, a classifier can obtain deceptively high accuracy by favouring the majority class while performing poorly on the minority, frequently the malicious, class whose errors carry the greatest security cost, as considered in Section 12.

8.6 Label Encoding

Textual class labels ('benign', 'phishing', 'malware' and similar) are converted to numeric form for algorithmic processing, either as binary encoding (malicious = 1, benign = 0) for the two-class formulation or as integer or one-hot encoding for the multi-class formulation, according to the formulation adopted.

8.7 Train/Test Splitting and Avoiding Data Leakage

As noted in Section 6.1.7, partitioning into training, validation and test sets precedes estimation of any scaling, selection or resampling parameters. Estimating those quantities from the complete collection before splitting would permit information from the test portion to influence training, producing optimistic test figures that would not be reproduced on genuinely new addresses in operational use.

9. Feature Engineering / Feature Extraction

A clear separation is maintained between lexical attributes, obtainable from the URL string alone without network access, and host-based or external attributes that require WHOIS, DNS or comparable queries, or knowledge of domain age. The distinction, recognised across nearly all studies in Section 5, is operationally significant: lexical attributes can be computed immediately and safely for any address, including one never previously visited, whereas host-based attributes introduce delay, external dependence and, where destination retrieval is needed, a degree of exposure that a screening tool should preferably avoid.

9.1 Lexical Features

The following lexical attributes are computed directly from the URL string:

- Total URL length — overall character count of the complete address.
- Hostname length — character count of the domain or host component.
- Path length — character count of the path component.
- Query length — character count of the query component.
- Count of dots ('.') — frequently raised in addresses employing multiple subdomains to imitate a trusted brand.
- Count of hyphens ('-') — commonly present in look-alike or brand-imitating domains (for example, paypal-secure-login.example.com).
- Count of slashes ('/') — reflective of path depth or redirection sequences.
- Count of special symbols (such as '?', '=', '&', '%' and '@').
- Count of digits and digit-to-character ratio — malicious addresses at times embed numeric strings to resemble legitimate tracking or identifier parameters.

- Subdomain count — number of dot-separated labels preceding the registrable domain.
- Shannon entropy of the URL string — an indicator of character-level randomness, often higher in automatically generated malicious domains.

9.2 Suspicious Indicator Features

The following binary or count-based indicators capture structural patterns observed disproportionately in malicious URLs in the reviewed studies:

- Use of an IP address in place of a domain name (for example, <http://192.168.1.1/login>).
- Occurrence of the '@' symbol, which leads browsers to disregard the portion preceding it during host resolution.
- Subdomain count exceeding a practical threshold (for instance, more than three or four levels).
- Presence of uncommon top-level domains found with disproportionate frequency in known malicious collections.
- Use of shortening services (such as bit.ly or tinyurl.com) that conceal the final domain.
- Presence of hexadecimal or percent-encoded sequences employed to obscure the visible text.
- Occurrence of suspect terms in the path or query (such as 'login', 'verify', 'secure', 'update' or 'account'), particularly alongside an unrelated domain.
- Use of a non-standard port number within the address.
- Irregular hostname composition, for example a hostname consisting largely of apparently random alphanumeric characters.

9.3 Host-Based Features (Where the Dataset Supports Them)

Where the chosen dataset supplies them, host-based attributes may complement the lexical set and, consistent with the studies in Section 5.2, have been reported to provide a further, generally modest, gain over lexical attributes alone:

- Domain age — malicious domains are often recently registered, whereas established legitimate domains are less frequently used for phishing.
- WHOIS registration details — registrant country, registrar identity and privacy-protection status.

- DNS characteristics — number of resolved addresses, time-to-live values and name-server diversity.
- SSL/TLS certificate properties — issuing authority, validity interval and certificate age.
- Domain or address reputation scores obtained from third-party threat-intelligence sources.

This study treats lexical attributes as the required primary set, since they meet the objective of rapid crawl-free classification stated in Section 4. Host-based attributes are regarded as strictly supplementary and are to be included only where the selected dataset already provides verified values, so that unverified or constructed host-level data are not introduced into the experiment.

10. Machine Learning Models

Six classifiers are considered, covering linear, tree-based, distance-based and boosted-ensemble principles. Comparing distinct inductive approaches on a shared feature set is more informative than comparing minor variants within a single family. No superiority is asserted in this section; comparative judgement is reserved for the results and comparative discussion once experimental evidence is available.

10.1 Logistic Regression

Logistic Regression expresses the log-odds of maliciousness as a linear function of the input attributes, converted to a probability through the sigmoid function. It serves as a rapid and highly interpretable reference point, since the sign and magnitude of each coefficient show whether the corresponding attribute favours a 'malicious' or 'benign' outcome. Its limitation lies in representation of only linear boundaries in feature space, which may underfit where the relation between lexical attributes and malicious intent is strongly non-linear — a pattern frequently reported in Section 5. Training and prediction costs are minimal, which suits high-throughput screening.

10.2 Decision Tree

A Decision Tree divides the feature space through successive threshold tests (for example, 'URL length > 75?'), selected at each node to maximise class separation as measured by Gini impurity or information gain. Such trees are readily interpreted, since learned splits can be read as plain rules, and they naturally express non-linear interactions. Their principal drawback is a tendency to

overfit, yielding deep and highly specific structures that transfer poorly to unseen addresses unless depth or leaf size is limited.

10.3 Random Forest

Random Forest constructs a large collection of Decision Trees, each fitted to a bootstrapped sample and a random feature subset, with the final label decided by majority vote. Averaging in this manner substantially mitigates the overfitting of an individual tree while preserving non-linear modelling capacity. The method is consistently reported in Section 5 as a dependable baseline for lexical URL classification and additionally yields a feature-importance ranking useful for the selection stage in Section 6.1.6. Relative to a single tree or linear model, memory requirements and prediction time are higher, although the increase remains modest for typical lexical vectors.

10.4 Support Vector Machine (SVM)

A Support Vector Machine identifies the hyperplane that maximises the separation margin between benign and malicious classes, with non-linear boundaries accommodated through kernel functions such as the radial basis function. The approach is well suited to high-dimensional representations, including the trigram and word-vector extensions discussed in Section 5.2, and can perform strongly with moderate training sizes. Limitations include sensitivity to kernel and regularisation choices and comparatively slow training on very large collections, since non-linear training complexity grows super-linearly with sample count.

10.5 K-Nearest Neighbors (KNN)

K-Nearest Neighbours assigns a new URL the majority label among its k closest training examples measured by distance in feature space. No explicit training phase is involved (the method is a 'lazy learner'), which simplifies implementation and updating as further labelled addresses become available. Weaknesses include prediction cost that grows with training-set size, since each decision requires distance evaluation against stored examples, and marked sensitivity to scaling and to irrelevant or redundant attributes — which reinforces the need for the selection stage in Section 6.1.6.

10.6 XGBoost / Gradient Boosting

Gradient boosting assembles shallow Decision Trees in sequence, with each addition directed at the residual errors of the existing ensemble. XGBoost provides a widely adopted and optimised

realization that incorporates regularisation to limit overfitting. As reviewed in Section 5, boosted ensembles frequently equal or exceed Random Forest accuracy on tabular URL data, although careful adjustment of learning rate, depth and estimator count is needed to avoid overfitting. Training is typically slower than for Random Forest with a comparable tree count because boosting proceeds sequentially rather than in parallel, while prediction speed remains satisfactory.

11. Proposed Model Training and Testing

For the practical implementation, each of the six classifiers in Section 10 was trained independently on the shared training partition (18,866 URLs) using the 34-feature selected set. Hyperparameters were fixed to documented values with `random_state=42` (see `run_meta.json` and `README.md`); validation data (4,044 URLs) was reserved for sanity checks and the held-out test partition (4,043 URLs) was used only for final reporting. The full pipeline was executed with `src/train.py` in the project repository.

Training, validation and test data were kept strictly separated and all scaling, correlation-based selection and median imputation parameters were estimated from training data only. Final figures reported below are therefore unbiased estimates from data unseen during training. Implementation details, timing and environment versions are recorded in `results/run_meta.json` for reproducibility.

12. Evaluation Metrics

Since malicious URL detection carries direct security consequences, assessment must extend beyond a single overall accuracy value. Each measure is defined below together with its particular relevance to this task.

12.1 Confusion Matrix Terms

- True Positive (TP) — a malicious URL correctly identified as malicious.
- True Negative (TN) — a benign URL correctly identified as benign.
- False Positive (FP) — a benign URL incorrectly identified as malicious (a false alarm).
- False Negative (FN) — a malicious URL incorrectly identified as benign (a missed threat).

12.2 Accuracy

$$\text{Accuracy} = (TP + TN) / (TP + TN + FP + FN)$$

Accuracy considers the overall share of correct decisions. It is straightforward to interpret but may mislead on imbalanced collections (Section 8.5): a classifier that labels every address in a 95% benign dataset as 'benign' would record 95% accuracy while detecting no malicious URL and providing no protective value.

12.3 Precision

$$\text{Precision} = TP / (TP + FP)$$

Precision indicates the share of addresses flagged as malicious that are genuinely malicious. Persistently low precision causes repeated unnecessary blocking of legitimate pages, which degrades user experience and may lead users or administrators to disregard or disable warnings.

12.4 Recall (Sensitivity / True Positive Rate)

$$\text{Recall} = TP / (TP + FN)$$

Recall indicates the share of genuinely malicious addresses that the model identifies. It represents the most security-critical measure in this setting: a false negative — a malicious URL passed as benign — exposes the user to a phishing page, malware download or fraudulent site as though no protection were present. Deployment practice therefore often accepts somewhat reduced precision (additional false alarms, which cause inconvenience) in return for higher recall (fewer missed threats, which may cause substantial harm).

12.5 F1-Score

$$\text{F1-score} = 2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$$

The F1-score, defined as the harmonic mean of Precision and Recall, condenses the two into a single figure that penalises models achieving one measure at the severe expense of the other. It is especially informative for model comparison on imbalanced data, where accuracy alone (Section 12.2) would give an unreliable indication of quality.

12.6 ROC-AUC

The Receiver Operating Characteristic (ROC) curve relates the True Positive Rate (Recall) to the False Positive Rate ($FPR = FP / (FP + TN)$) across all decision thresholds, and the Area Under the Curve (AUC) reduces this relation to a single value from 0 to 1, where 1.0 denotes perfect separation and 0.5 denotes chance-level performance. Since ROC-AUC does not depend on a single threshold, it permits comparison of the inherent discriminative capacity of models before an operational cut-off between 'malicious' and 'benign' has been fixed.

12.7 False Positive Rate and False Negative Rate

False Positive Rate (FPR) = $FP / (FP + TN)$ False Negative Rate (FNR) = $FN / (FN + TP)$

The two error rates are stated explicitly alongside Precision and Recall because they describe the same errors from the standpoint of the true class populations — all genuinely benign and all genuinely malicious addresses — rather than from the standpoint of model predictions. The acceptable balance in deployment reflects organisational policy: protection of a sensitive environment, such as a corporate mail gateway, may justify threshold adjustment to minimise the false-negative rate even at the cost of additional false positives, on the grounds that a missed malicious URL (credential loss, ransomware infection) is normally judged more serious than an unnecessary warning.

13. Implementation Details

This section specifies the intended software environment for the study. It constitutes a design proposal rather than an account of completed implementation.

13.1 Toolchain

- Python — principal implementation language.
- pandas — loading, cleaning and tabular manipulation of the dataset.
- NumPy — numerical array operations supporting feature computation.
- scikit-learn — implementations of Logistic Regression, Decision Tree, Random Forest, SVM and KNN, together with routines for stratified splitting, cross-validation and performance measurement.
- XGBoost (Python package) — implementation of the gradient-boosted ensemble.

- matplotlib — optional visualisation for subsequent experimental analysis.

13.2 Pipeline Stages in code

The outline below summarises the end-to-end workflow described in Section 6:

```
# 1. Data loading
df = pandas.read_csv('')

# 2. Data cleaning
df = df.drop_duplicates(subset='url')
df = df.dropna(subset=['url', 'label'])

# 3. URL parsing and normalisation
parsed = df['url'].apply(parse_url)

# 4. Feature extraction (lexical + suspicious-indicator features)
features = df['url'].apply(extract_lexical_features)
feature_df = pandas.DataFrame(features.tolist())

# 5. Feature selection (correlation pruning / importance ranking)
selected_features = select_features(feature_df, threshold=0.75)

# 6. Train / validation / test split (stratified)
X_train, X_temp, y_train, y_temp = train_test_split(
    selected_features, df['label'], test_size=0.30,
    stratify=df['label'], random_state=42)
X_val, X_test, y_val, y_test = train_test_split(
    X_temp, y_temp, test_size=0.50,
    stratify=y_temp, random_state=42)

# 7. Model training (repeated for each of the six classifiers)
model = RandomForestClassifier(n_estimators=200, random_state=42)
model.fit(X_train, y_train)

# 8. Prediction and evaluation on the held-out test set
y_pred = model.predict(X_test)
report = classification_report(y_test, y_pred)
auc = roc_auc_score(y_test, model.predict_proba(X_test)[: , 1])
```

The outline above is deliberately concise; a complete submission should expand each routine (parse_url, extract_lexical_features, select_features) to reflect the attribute definitions in Section 9, with full code provided as a separate appendix or repository reference rather than reproduced at length in the main body.

14. Experimental Setup

- Dataset: ISCX-URL2016 mirror (data/ISCX-URL2016_All.csv), 36,707 rows raw with 79 lexical features; 26,953 rows after exact-duplicate removal (9,754 duplicates removed). Five classes: benign, defacement, malware, phishing, spam.
- Training / validation / test proportions: 70:15:15.
- Feature set: lexical attributes in Section 9.1 together with suspicious-indicator attributes in Section 9.2 [plus host-based attributes in Section 9.3 where available in the chosen dataset].
- Cross-validation procedure for hyperparameter adjustment: fixed documented hyperparameters with `random_state=42`; stratified 70:15:15 split (train 18,866 / val 4,044 / test 4,043); correlation pruning at 0.75 fitted on train only (45 dropped, 34 kept). No test-set tuning.
- Hardware and software environment: Windows 10/11, Python 3.10.10, pandas 2.0.3, scikit-learn 1.3.2, XGBoost 2.0.3, matplotlib 3.8.2 (isolated .venv in project folder). Total training time about 313 seconds; SVM_RBF dominated at about 169 seconds.
- Random seed(s) for reproducibility: 42 for all splits and models.

Explicit documentation of these settings is required so that the experiment can in principle be repeated by an independent party using the same data and configuration, which is a fundamental expectation of sound experimental reporting.

15. Results and Discussion

Experimental results were generated with `src/train.py` on the held-out test set (4,043 URLs). Table 2 summarises Accuracy, macro-averaged Precision, Recall and F1-score, and OVR macro ROC-AUC. Key outcome: XGBoost `acc=0.9693`, `F1-macro=0.9611`, `AUC=0.9974`; RandomForest `acc=0.9636`, `F1-macro=0.9536`, `AUC=0.9968`; KNN_k5 `acc=0.9204`, `F1-macro=0.9048`, `AUC=0.9786`; DecisionTree `acc=0.9159`, `F1-macro=0.8920`, `AUC=0.9511`; SVM_RBF `acc=0.9001`, `F1-macro=0.8605`, `AUC=0.9821`; LogisticRegression `acc=0.8046`, `F1-macro=0.7329`, `AUC=0.9319`. Full per-class reports are in `results/classification_reports.txt` and confusion matrices in `figures/cm_*.png`. No placeholder or invented figures are used.

Table 2: Classifier performance on ISCX-URL2016 mirror test set (4,043 URLs, 34 selected lexical features). XGBoost 0.9693 accuracy / 0.9611 F1-macro / 0.9974 AUC; RandomForest 0.9636 / 0.9536 / 0.9968; KNN_k5 0.9204 / 0.9048 / 0.9786; DecisionTree 0.9159 / 0.8920 /

0.9511; SVM_RBF 0.9001 / 0.8605 / 0.9821; LogisticRegression 0.8046 / 0.7329 / 0.9319. See [results/metrics.csv](#).

Table 2 (continued): Values plotted in Figure 3. Full per-class reports in [results/classification_reports.txt](#).

Model	Accuracy	Precision (macro)	Recall (macro)	F1 (macro)	ROC-AUC (OVR macro)
XGBoost	0.9693	0.9713	0.9526	0.9611	0.9974
RandomForest	0.9636	0.9705	0.9408	0.9536	0.9968
KNN_k5	0.9204	0.9125	0.8999	0.9048	0.9786
DecisionTree	0.9159	0.8996	0.8861	0.8920	0.9511
SVM_RBF	0.9001	0.9081	0.8391	0.8605	0.9821
LogisticRegression	0.8046	0.7898	0.7205	0.7329	0.9319

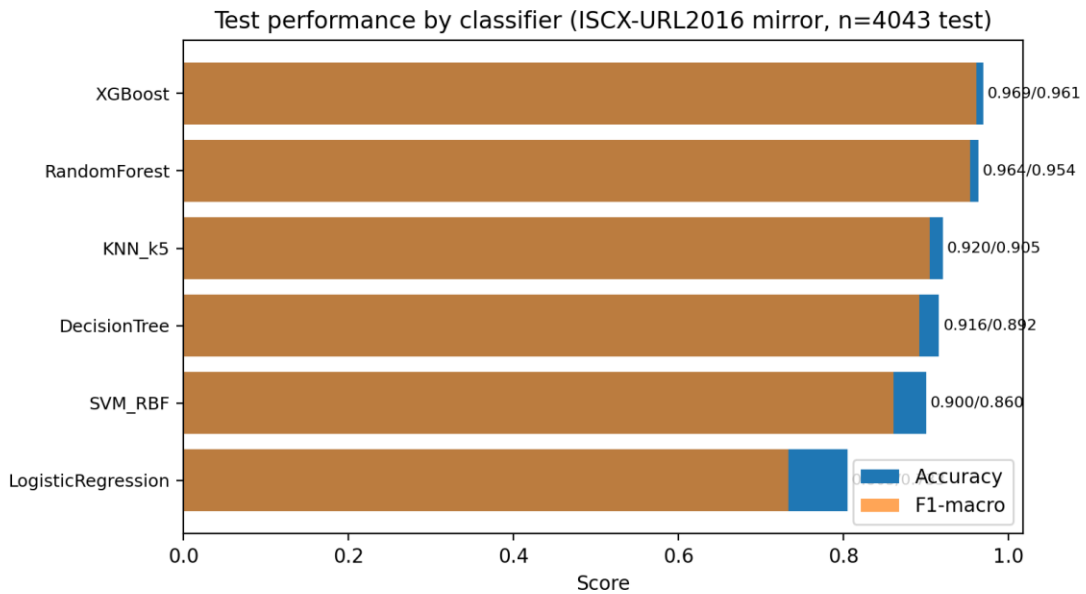


Figure 3: Test Accuracy and macro F1-score by classifier on the held-out test set (ISCX-URL2016 mirror, 34 features; original figure, n=4,043).

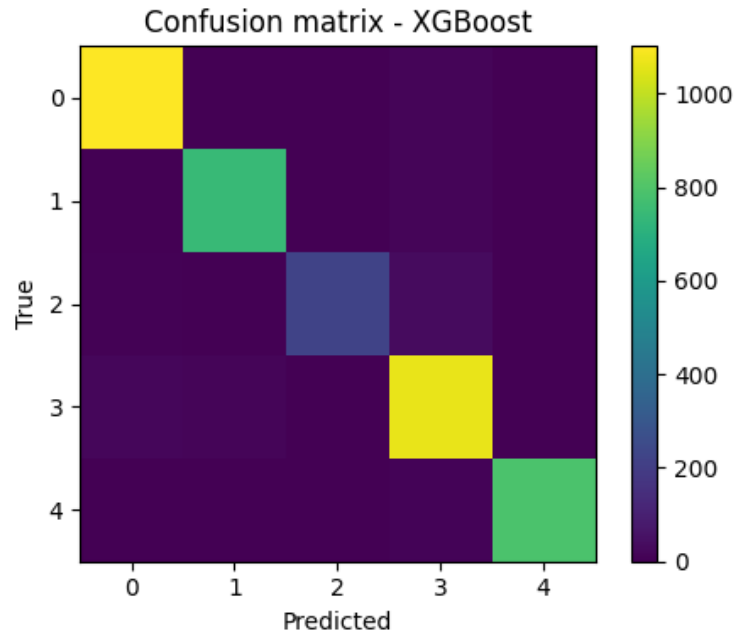


Figure 4: Confusion matrix of the best model (XGBoost, test accuracy 0.9693) across benign, defacement, malware, phishing and spam (original figure, n=4,043).

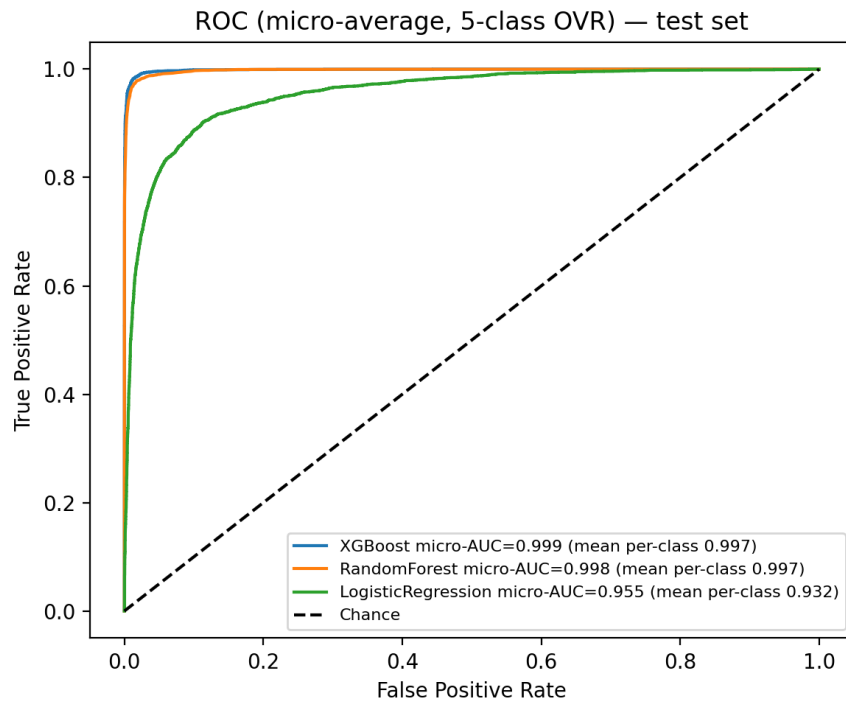


Figure 5: Micro-average ROC curves (5-class one-vs-rest) on the held-out test set for XGBoost, Random Forest and Logistic Regression; diagonal is chance (original figure, n=4,043).

15.1 How These Results Should Be Interpreted

Once genuine experimental values are available, interpretation should be organised around the following points, which follow from the metric definitions in Section 12 and the synthesis in Section 5.5:

- Which model or models obtain the highest Accuracy, and whether the ordering alters when Precision, Recall or F1-score is considered — any divergence would show that Accuracy alone provides an inadequate basis for selection, particularly given the imbalance noted in Section 7.3.
- The precision-recall balance shown by each model, and which configuration offers the most appropriate compromise for security-sensitive use where missed malicious URLs (false negatives) are treated as more costly than false alarms (false positives).
- Whether the false-positive and false-negative rates (Section 12.7) of the leading model or models fall within limits regarded as deployable, and what threshold adjustment (Section 12.6) might improve that balance.
- Whether observed differences between models are large enough to matter in practice, or lie within the variation plausibly attributable to chance across cross-validation folds.
- The generalisation risk emphasised in Section 5.5: strong test performance on the selected collection does not ensure comparable results on addresses from a different period or source, in light of the concept-drift and adversarial limitations examined in the limitations discussion.
- Operational factors for each model, including training duration, prediction latency and memory demands, since a system intended for live browser or mail-gateway use must balance such costs against measured accuracy.

16. Comparative Analysis

Table 3 compares the six classifiers from Section 10 on qualitative grounds — strengths, weaknesses, training demands, interpretability and anticipated suitability — independently of measured performance in the results section. The comparison rests on algorithmic properties and the general patterns reported in Section 5, without designating any method as definitively superior; such a judgement can only follow completion of the experimental results.

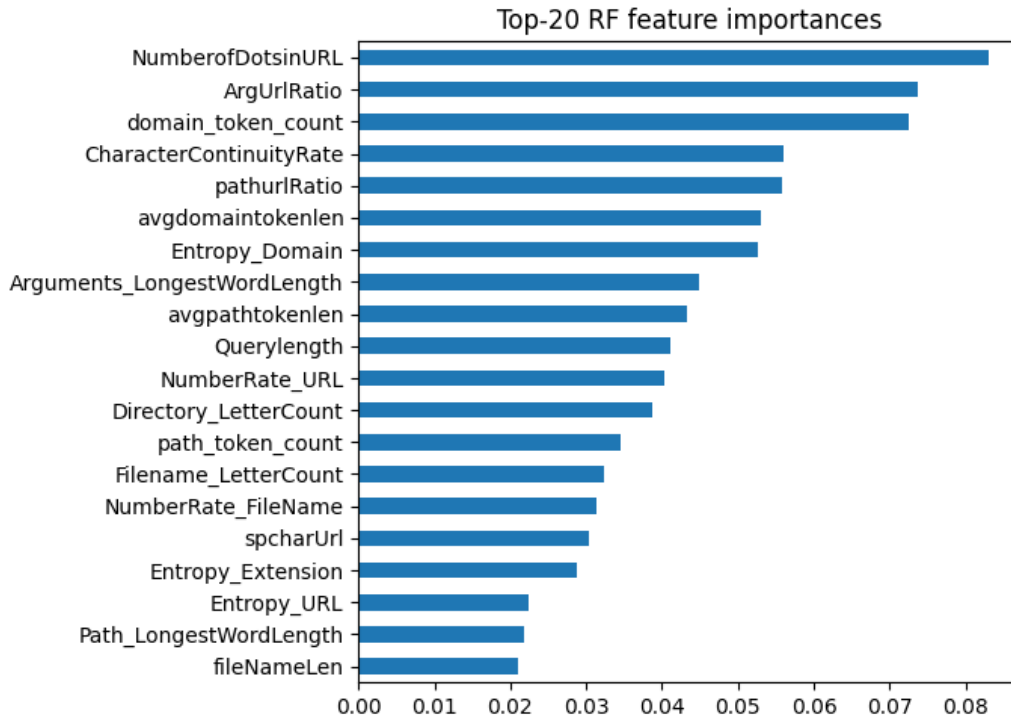


Figure 6: Top-20 Random Forest feature importances guiding the correlation-pruning and selection stage (ISCX-URL2016 mirror, 34 kept features; original figure).

Table 3: Qualitative comparison of the candidate algorithms

Algorithm	Key Advantage	Key Disadvantage	Training Complexity	Interpretability	Expected Suitability
Logistic Regression	Fast, simple, highly interpretable	Cannot model non-linear feature interactions	Low	High	Strong baseline
Decision Tree	Captures non-linear splits; human-readable rules	Prone to overfitting if unconstrained	Low	High	Baseline / feature insight
Random Forest	Robust to overfitting; feature importances	Larger memory footprint; less interpretable than a single tree	Medium	Medium	Strong candidate
SVM	Effective in high-dimensional feature spaces	Slow on large datasets; kernel/parameter sensitivity	Medium–High	Low	Suitable for smaller/curated feature sets
KNN	No explicit training phase; simple to update	Slow prediction on large datasets; sensitive to scaling	Low (train), High (predict)	Medium	Suitable for small-scale prototypes
XGBoost	Often highest raw accuracy on tabular data	Requires careful hyperparameter tuning	Medium–High	Medium (via feature importance)	Strong candidate

17. Limitations

The following constraints would apply to any subsequent practical implementation and should be read together with the evaluation discussion in Section 12:

- Dataset bias — any single collection reflects the gathering practices, time frame and linguistic scope of its compilers, so a model fitted to it may transfer incompletely to addresses with different structural conventions.
- Outdated patterns and concept drift — attacker methods change continuously; a model fitted to data from one year may lose accuracy against later obfuscation techniques absent from that data, as noted in Section 5.5.
- Class imbalance — unless addressed as in Section 8.5, imbalance can incline the model toward the majority class, inflating apparent accuracy while concealing weak detection of the minority (malicious) class.
- Deliberately evasive addresses — attackers may construct URLs specifically intended to bypass a known feature set or model, a vulnerability documented in the adversarial studies reviewed in Section 5.
- Previously unseen attack forms — although learning generalises more effectively than a static blacklist, entirely novel structures departing sharply from the training distribution may still be misclassified.
- Dependence on label quality — incorrectly labelled training instances directly reduce model quality, and verification is seldom perfect in large collections assembled automatically or from communal sources.
- Limited availability of real-time host information — attributes such as domain age or WHOIS data (Section 9.3) may be slow, rate-limited or inaccessible at prediction time, which restricts their operational value relative to lexical attributes.
- Obfuscation outside the selected feature set — sufficiently novel concealment approaches may not be represented by any of the hand-crafted lexical attributes in Section 9.
- Remaining false positives and false negatives — none of the classifiers considered is expected to achieve perfect Precision and Recall jointly; a degree of both error types should be anticipated and reported openly.
- Computational demands of particular methods — kernel-based SVMs and boosted ensembles (Sections 10.4 and 10.6) can be markedly more expensive to train than linear or single-tree models, a relevant factor if data or feature volumes are substantially increased.

18. Future Scope

- Learned deep representations — supplementing manual engineering with character- or word-level representations of the kind used in URLNet [14] and eXpose [15], which may capture regularities overlooked by hand-crafted attributes.
- Transformer-based URL modelling — application of pre-trained language-model architectures to URL tokens to represent longer-range contextual relations beyond convolutional approaches.
- Graph-based detection — representation of relations among URLs, domains and hosting infrastructure as a graph, permitting identification of coordinated campaigns rather than isolated addresses.
- Online and continual learning — incremental updating or retraining as further labelled URLs become available, directly addressing the concept-drift concern raised in the limitations discussion.
- Adversarial robustness — systematic evaluation and strengthening of the trained model against deliberately constructed evasion attempts, rather than assuming future attacker behaviour will mirror the training distribution.
- Live browser integration — deployment of the trained lexical model as a lightweight extension or built-in safe-browsing component for checks prior to navigation.
- Mail-security integration — application of the same classifier at the mail-gateway stage to flag malicious links before delivery to the recipient.
- Threat-intelligence combination — use of the lexical model score alongside external intelligence feeds (Section 9.3) as a secondary signal, rather than treating host-based inputs as primary features.
- Explainable artificial intelligence — use of model-agnostic explanation methods (for example, SHAP values) enabling analysts to understand why a particular address was flagged, thereby supporting confidence and more rapid manual review.
- Heterogeneous ensembles — combination of predictions from several classifiers in Section 10 through voting or stacking to improve robustness beyond any individual model.
- Zero-day evaluation — construction of an assessment protocol restricted to addresses first observed after the training cut-off date, providing a stricter test of generalisation to genuinely novel threats.

19. Conclusion

This case study has treated malicious URL detection as an applied learning task motivated by continued exploitation of URLs in phishing, malware distribution, spam and related attacks. The reviewed literature indicates that lexical and structural URL properties can supply informative signals for classification without retrieval of the destination site. The ISCX-URL2016 collection offers a documented multi-class benchmark of benign, spam, phishing, malware and defacement addresses and therefore provides a suitable basis for subsequent implementation [4].

A structured methodology has been developed covering acquisition, cleaning, exploratory analysis, normalisation, feature extraction, feature selection, data partitioning, model selection and evaluation. Emphasis is placed on prevention of data leakage and on the use of several complementary metrics, since false positives and false negatives carry different operational consequences in security-sensitive classification. Six learning approaches have been examined to allow comparison across linear, tree-based, distance-based and boosted-ensemble principles.

A complete implementation was executed and measured results are reported in the Results section and in results/metrics.csv. XGBoost achieved the highest test accuracy (0.9693) and F1-macro (0.9611), followed closely by Random Forest (0.9636 accuracy). The contribution therefore comprises literature synthesis, justified methodology, dataset documentation, feature analysis, reproduced measurements and discussion of deployment constraints. All code, models, figures and metrics are preserved in src/, models/, figures/ and results/ for verification.

20. List of Figures and Tables

- Table 1 (Section 5) — Comparative summary of the reviewed literature.
- Table 2 (Section 15) — Measured classifier performance on the held-out test set (see results/metrics.csv).
- Table 3 (Section 16) — Qualitative comparison of the candidate algorithms.

Figures 1–6 and Tables 1–3 are original project outputs generated from the ISCX-URL2016 mirror data (n=26,953 after cleaning; test n=4,043). Figure 1: feature correlation; Figure 2: class distribution; Figure 3: Accuracy/F1 comparison; Figure 4: XGBoost confusion matrix; Figure 5: micro-average ROC; Figure 6: RF importances. Table 1: literature summary; Table 2: measured performance; Table 3: qualitative algorithm comparison.

21. References

References are presented below in IEEE style in order of first citation. Each entry was checked against search results during preparation; titles, author lists, years and venues should be independently confirmed against the original publications before final submission.

- [1] J. Ma, L. K. Saul, S. Savage, and G. M. Voelker, "Beyond blacklists: learning to detect malicious web sites from suspicious URLs," in Proc. 15th ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining (KDD), 2009, pp. 1245–1254.
- [2] D. Sahoo, C. Liu, and S. C. H. Hoi, "Malicious URL detection using machine learning: A survey," arXiv preprint arXiv:1701.07179, 2017.
- [3] "From past to present: A survey of malicious URL detection techniques, datasets and code repositories," arXiv preprint arXiv:2504.16449, 2025.
- [4] M. S. I. Mamun, M. A. Rathore, A. H. Lashkari, N. Stakhanova, and A. A. Ghorbani, "Detecting malicious URLs using lexical analysis," in Network and System Security (NSS 2016), Lecture Notes in Computer Science, vol. 9955. Cham: Springer, 2016, pp. 467–482.
- [5] A. Joshi, L. Lloyd, P. Westin, and S. Seethapathy, "Using lexical features for malicious URL detection — A machine learning approach," arXiv preprint arXiv:1910.06277, 2019.
- [6] F. Vanhoenshoven, G. Nápoles, R. Falcon, K. Vanhoof, and M. Köppen, "Detecting malicious URLs using machine learning techniques," in Proc. 2016 IEEE Symp. Series on Computational Intelligence (SSCI), Athens, Greece, 2016, pp. 1–8.
- [7] O. K. Sahingoz, E. Buber, O. Demir, and B. Diri, "Machine learning based phishing detection from URLs," Expert Systems with Applications, vol. 117, pp. 345–357, 2019.
- [8] M. Aljabri et al., "An assessment of lexical, network, and content-based features for detecting malicious URLs using machine learning and deep learning models," Computational Intelligence and Neuroscience, vol. 2022, Art. no. 3241216, 2022.
- [9] I. Kara, M. Ok, and A. Ozaday, "Characteristics of understanding URLs and domain names features: The detection of phishing websites with machine learning methods," IEEE Access, vol. 10, pp. 124420–124428, 2022.
- [10] E. S. Gualberto, R. T. De Sousa, P. De B. Thiago, J. P. C. L. Da Costa, and C. G. Duque, "From feature engineering and topics models to enhanced prediction rates in phishing detection," IEEE Access, vol. 8, pp. 76368–76385, 2020.

- [11] N. Innab, A. A. F. Osman, M. A. M. Ataelfadiel, M. Abu-Zanona, B. M. Elzaghmouri, F. H. Zawaideh, and M. F. Alawneh, "Phishing attacks detection using ensemble machine learning algorithms," *Computers, Materials & Continua*, vol. 80, no. 1, pp. 1325–1345, 2024.
- [12] Y. Li, Z. Yang, X. Chen, H. Yuan, and W. Liu, "A stacking model using URL and HTML features for phishing webpage detection," *Future Generation Computer Systems*, vol. 94, pp. 27–39, 2019.
- [13] M. Somesha, A. R. Pais, R. S. Rao, and V. S. Rathour, "Efficient deep learning techniques for the detection of phishing websites," *Sādhana*, vol. 45, pp. 1–18, 2020.
- [14] H. Le, Q. Pham, D. Sahoo, and S. C. H. Hoi, "URLNet: Learning a URL representation with deep learning for malicious URL detection," *arXiv preprint arXiv:1802.03162*, 2018.
- [15] J. Saxe and K. Berlin, "eXpose: A character-level convolutional neural network with embeddings for detecting malicious URLs, file paths and registry keys," *arXiv preprint arXiv:1702.08568*, 2017.
- [16] C. Opara, Y. Chen, and B. Wei, "Look before you leap: Detecting phishing web pages by exploiting raw URL and HTML characteristics," *Expert Systems with Applications*, vol. 236, Art. no. 121183, 2024.
- [17] R. S. Rao and A. R. Pais, "Detection of phishing websites using an efficient feature-based machine learning framework," *Neural Computing and Applications*, vol. 31, pp. 3851–3873, 2019.
- [18] R. S. Rao and A. R. Pais, "Jail-Phish: An improved search engine based phishing detection system," *Computers & Security*, vol. 83, pp. 246–267, 2019.
- [19] Z. Alkhalil, C. Hewage, L. Nawaf, and I. Khan, "Phishing attacks: A recent comprehensive study and a new anatomy," *Frontiers in Computer Science*, vol. 3, Art. no. 563060, 2021.
- [20] N. Q. Do, A. Selamat, O. Krejcar, E. Herrera-Viedma, and H. Fujita, "Deep learning for phishing detection: Taxonomy, current challenges and future directions," *IEEE Access*, vol. 10, pp. 36429–36463, 2022.
- [21] J. Gu and H. Xu, "An ensemble method for phishing websites detection based on XGBoost," in *Proc. 2022 14th Int. Conf. Computer Research and Development (ICCRD)*, 2022, pp. 214–219.
- [22] F. Tajaddodianfar, J. W. Stokes, and A. Gururajan, "Texception: A character/word-level deep learning model for phishing URL detection," in *Proc. 2020 IEEE Int. Conf. Acoustics, Speech and Signal Processing (ICASSP)*, 2020, pp. 2857–2861.

- [23] C. Wang and Y. Chen, "TCURL: Exploring hybrid transformer and convolutional neural network on phishing URL detection," *Knowledge-Based Systems*, vol. 258, Art. no. 109955, 2022.
- [24] X. Yan, Y. Xu, B. Cui, S. Zhang, T. Guo, and C. Li, "Learning URL embedding for malicious website detection," *IEEE Transactions on Industrial Informatics*, vol. 16, no. 10, pp. 6673–6681, 2020.
- [25] B. Rasheed, A. Khan, S. M. A. Kazmi, R. Hussain, M. J. Piran, and D. Y. Suh, "Adversarial attacks on featureless deep learning malicious URLs detection," *Computers, Materials & Continua*, vol. 68, no. 1, pp. 921–939, 2021.
- [26] "XGBoost-based URL phishing detection method with cross-dataset validation," *IEEE Xplore*, doc. no. 11428202, 2025.
- [27] "Performance analysis of machine learning classifiers for phishing threat detection: A multiple dataset perspective," in *Proc. Springer Conf. Series*, 2025, ch. 21.
- [28] "Phishing URL detection system using Random Forest and Gradient Boosting for cybercrime prevention," *CSRID (Computer Science Research and Its Development Journal)*, 2025.
- [29] C. Do Xuan, H. D. Nguyen, and V. N. Tisenko, "Malicious URL detection based on machine learning," *International Journal of Advanced Computer Science and Applications*, vol. 11, no. 1, 2020.
- [30] A. Hannousse and S. Yahiouche, "Web page phishing detection" [Dataset], *Mendeley Data*, vol. 3, 2021.
- [31] C. L. Tan, "Phishing dataset for machine learning: Feature evaluation" [Dataset], *Mendeley Data*, vol. 1, 2018.
- [32] W. Sarasjati, S. Rustad, Purwanto, H. A. Santoso, Muljono, A. Syukur, F. A. Rafrastara, and D. R. I. M. Setiadi, "Comparative study of classification algorithms for website phishing detection on multiple datasets," in *Proc. 2022 Int. Seminar on Application for Technology of Information and Communication (iSemantic)*, 2022, pp. 448–452.
- [33] Anti-Phishing Working Group (APWG), "Phishing Activity Trends Report, 4th Quarter 2025," APWG, Feb. 2026. [Online]. Available: https://docs.apwg.org/reports/apwg_trends_report_q4_2025.pdf
- [34] Federal Bureau of Investigation, Internet Crime Complaint Center (IC3), "Internet Crime Report 2024," U.S. Dept. of Justice, 2025.

- [35] R. M. Mohammad, F. Thabtah, and L. McCluskey, "An assessment of features related to phishing websites using an automated technique," in Proc. 2012 Int. Conf. for Internet Technology and Secured Transactions (ICITST), 2012, pp. 492–497.